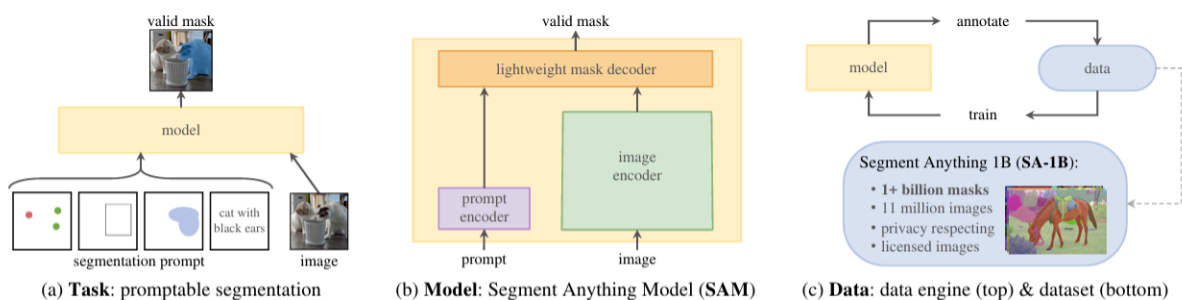


SAM

Segment Anything

1 Introduction



기존 문제

객체 분할(Segmentation): 컴퓨터 비전에서 매우 중요한 작업, but

- 대부분의 모델은 특정 데이터셋이나 특정 도메인에서 학습, 새 도메인에 적용하려면 다시 학습하거나 많은 라벨링 작업 필요
- ex:
 - 의료 영상에 특화된 모델은 일반 사진에서 사람이나 사물을 잘 못 자름
 - 자율주행용 모델은 위성 영상에서 건물 경계를 잘 못 찾음

SAM

Segment Anything: 모든 객체를 한 번에, 즉시 분할하는 범용 모델

핵심:

1. **Zero-shot 성능:** 학습 데이터에 없던 객체/환경에서도 잘 동작
2. **대규모 데이터 학습:** 방대한 양의 이미지와 마스크를 이용해 범용성 확보
3. **Promptable Segmentation:** 사용자가 클릭, 박스, 텍스트 등으로 관심 객체 지정 → 즉시 마스크 생성

Promptable Segmentation?

기존 segmentation 모델: 고정된 목표 학습 (ex. 사람 세그먼트만 하는 모델, 고양이만 분할하는 모델)

SAM: 사용자가 원하는 대상을 프롬프트(prompt)로 알려주면 해당 객체만 분할

- 프롬프트 예시:
 - 점(Point): 여기 클릭 → 이 객체 자르기
 - 박스(Box): 이 영역 안에 있는 객체 자르기
 - 텍스트(Text): 고양이 하면 고양이만 분할(실험적으로 가능)

왜 중요?

한 모델로 다양한 작업 수행 가능:

- 단일 객체 분할
- 다중 객체 분할
- 배경 제거
- 데이터 라벨링 보조

도메인 제약 없이 다양한 산업에 바로 적용 가능:

- 의료 → 병변 분할
- 위성 → 건물, 도로 추출
- 자율주행 → 차량, 보행자 인식

대규모 데이터셋 SA-1B

SAM이 가능해진 이유 중 하나는 SA-1B 데이터셋 덕분!

- 규모: 이미지 1억 1천만 장, 마스크 11억 개
- 수집 과정: 초기 모델 + 사람 검수 → 점점 자동화 → 대규모 확장

⇒ 기존의 특정 데이터셋 한계 넘어 범용성 확보

2 Segment Anything Task

2.1 기존 segmentation

고정된 출력 목표

- Semantic segmentation: 모든 픽셀을 사전 정의된 클래스 중 하나로 할당
- Instance segmentation: 사전 정의된 클래스의 객체별 마스크 생성

SAM의 **Segment Anything Task**: 사용자가 주는 **프롬프트**에 맞춰 어떤 객체든 **segmentation** 하는 새로운 문제 정의

2.2 Promptable Segmentation

핵심 = **프롬프트(prompt)**: 사용자가 지정하는 입력 신호

- **Point prompt**: 특정 위치 클릭
- **Box prompt**: 사각형 범위 지정
- **Mask prompt**: 기존 마스크를 기반으로 수정

모델은 **프롬프트에 맞는 마스크** 즉시 출력,

프롬프트가 바뀌면 **동일 이미지에서도 다른 객체** 바로 분할 가능!

2.3 Zero-shot

모델이 사전에 본 적 없는 객체나 환경에서도 잘 동작

ex:

- 학습 데이터에 없던 희귀 동물 사진 → 프롬프트 클릭 하나로 분할
- 의료 이미지, 위성 사진 등 전혀 다른 도메인에도 적용 가능

2.4 출력 형식

단일 프롬프트에 대해 **여러 후보 마스크** 반환 : 보통 3개 마스크와 각 마스크의 **confidence score** 제공

(confidence score: 이게 얼마나 맞다고 생각하는지 확률 비슷한 값)

왜 여러 개 출력? 프롬프트의 모호성 때문!

같은 클릭이라도 사람이 원하는 영역이 다를 수 있음



- ex. "이 점"을 클릭했을 때

1. 점이 찍힌 작은 물체만 선택
2. 그 물체 주변까지 포함한 영역 선택
3. 배경 일부까지 포함한 더 넓은 영역 선택

⇒ SAM은 이런 가능성을 다 고려해서 **3개의 후보 마스크**를 만드는 것

사용자는 이 중 원하는 것 선택 가능

구조

1. Mask Decoder가 한 번의 입력에서 **여러 Query**를 동시에 처리
2. 각 Query는 조금씩 다른 경계 조건을 가정하고 마스크 생성
3. 이렇게 나온 마스크마다 **Confidence Score**(이게 얼마나 맞다고 생각하는지 확률 비슷한 값)을 붙임

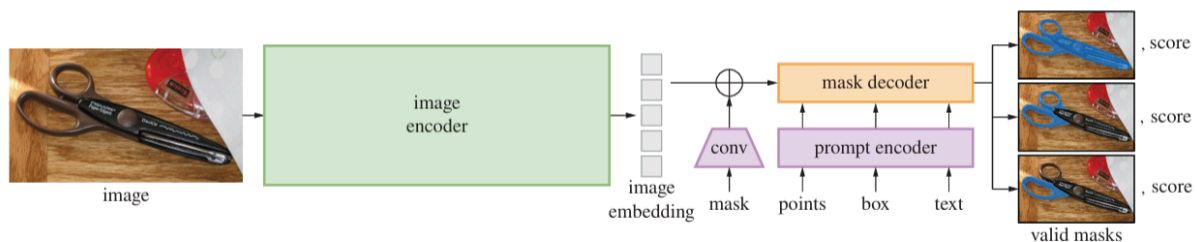
장점

1. **사용자 선택권**: 사용자가 원하는 마스크를 클릭해서 선택 가능
2. **자동 선택 가능**: 앱에서 가장 점수가 높은 마스크를 자동 채택 가능
3. **라벨링 효율성↑**: 데이터 라벨링 도구에서 한 번 클릭으로 여러 안을 제공해, 수정량 최소화

2.5 이 task 왜 필요?

- **범용성**: 한 번 학습된 모델이 모든 도메인에서 사용 가능
- **상호작용성**: 사용자가 원하는 대로 세밀하게 조정 가능
- **데이터 제작 자동화**: 새로운 데이터셋 라벨링을 빠르게 생성할 수 있음
- **속도**: 브라우저에서 실시간 반응 가능 → 편집 툴, AR/VR, 로봇 비전 등에 즉시 활용 가능

3 Segment Anything Model



SAM은 크게 세 개로 나눌 수 있음:

Image Encoder → Prompt Encoder → Mask Decoder

이미지는 무거운 인코더를 딱 한 번 거쳐 이미지 임베딩 되고 → 이후 가벼운 프롬프트/디코더가 이 임베딩을 빠르게 질의해서 마스크를 뽑아내는 구조

사용자의 프롬프트가 애매할 수 있어서 한 번에 여러 후보 마스크(보통 3개) + 신뢰도(예상 IoU)를 같이 생성

1) Image Encoder

- 백본: **ViT-H/16** (MAE 사전학습 + 고해상도 입력을 위한 약간의 수정)
- 입력 이미지를 1024×1024로 맞추고 임베딩 맵(채널 256, 공간 64×64)으로 변환
- 1×1 conv → 3×3 conv → LayerNorm으로 채널 256 정리

이걸 **한 번만 계산하고, 이후 프롬프트가 바뀌어도 계속 재사용함**

⇒ 무거운 일(이미지 이해)은 1번만, 프롬프트에 따른 세부 분할은 가볍게 N번

2) Prompt Encoder: 점/박스/마스크/텍스트 → 벡터

프롬프트는 크게 **Sparse**(점, 박스, 텍스트)와 **Dense**(마스크)로 나뉨

점(point)

: 좌표 positional encoding + 전경 or 배경 학습 임베딩 합 → 256차원 토큰

1. 사용자 클릭 → 좌표 (x=150, y=300)
2. 이 좌표를 좌표용 인코딩(positional encoding) → 좌표가 벡터로 변환
3. 이 점이 전경(포함)인지 배경(제외)인지 따로 표시하는 숫자 더함
4. 256차원짜리 토큰 하나가 만들어짐

박스(box)

: 좌상단·우하단 모서리를 각각 별도 토큰으로(좌표 pos enc + corner 종류 임베딩) → 256차원 토큰 2개

1. 네모는 두 꼭짓점 좌표(좌상단, 우하단)로 표현 가능
2. SAM은 두 꼭짓점 각각을 **좌표 인코딩 + 꼭짓점 종류 표시**로 벡터화
3. 박스 하나당 토큰 2개(256차원씩)가 만들어짐

텍스트

: CLIP의 텍스트 인코더 임베딩 사용(확장 실험) → 256차원 공간에 맞춰 통합

1. 사용자가 "the cat" 같은 설명을 쓸 수도 있지
2. 이건 CLIP이라는 텍스트 인코더로 문장을 숫자 벡터로 변환해야 됨
3. 변환된 벡터를 SAM이 쓰는 256차원 공간에 맞게 조정해서 사용



CLIP?

Contrastive Language-Image Pretraining

이미지랑 문장을 같이 이해하는 뇌 같은 모델

ex. 이미지 1 + "검은 고양이"라는 문장 넣으면 → 둘을 같은 숫자 공간(벡터 공간)에 배치 →

사진 벡터와 문장 벡터가 서로 가깝게 되도록 학습

왜 SAM에서 CLIP을 쓰나?

SAM이 텍스트로도 프롬프트 받게 하려면 문장을 숫자로 바꾸는 방법이 필요
⇒ CLIP은 이미 문장 → 숫자 벡터 변환을 잘하는 모델이니까 가져다 쓰는 것!

어떻게 숫자로 바꿈?

1. 문장(a cat) → CLIP의 텍스트 인코더에 넣음
2. CLIP이 이 문장을 **512차원** 정도의 벡터로 변환 (이 벡터는 '고양이'라는 개념 담음)
3. 그런데 SAM은 모든 입력 벡터를 **256차원** 크기로 쓰기로 했잖아
→ 그래서 중간에 변환(Projection) 층을 넣어 512 → 256으로 줄여야 됨
4. 그럼 이제 "a cat"이라는 텍스트도 256차원 숫자 벡터(토큰)로 변환된 상태
5. 이 벡터는 **Prompt Encoder**에서 이미지 특징과 합쳐져 → Mask Decoder로 들어가서 → '고양이'에 해당하는 마스크를 만들 수 있음

비유

CLIP = **통역사**

→ 사람 말(문장)을 기계가 알아듣는 **숫자 언어**로 번역

SAM = **화가**

→ 번역된 숫자 언어를 듣고 그림에서 그 부분을 찾아 색칠

마스크(dense)

: SAM은 사진을 한 번 분석해서 사진 지도(이미지 임베딩)를 만들어 두는데, 관심 있는 영역 표시된 마스크를 주면 그 부분에 집중해야 하는구나 하고 참고하게 됨

= 사진 분석에 힌트를 주는 보조 지도

1. **마스크 축소**: 계산을 줄이기 위해 원본 마스크를 1/4 크기로 줄임 ($1024 \times 1024 \rightarrow 256 \times 256$)
2. **특징 추출(합성곱, Conv)**
 - 축소된 마스크에서 모양, 위치 정보 추출
 - 합성곱 필터(2×2 크기, 한 번에 2픽셀씩 이동) 2번 적용
 - 이 과정에서 마스크의 채널 수를 늘려서 더 많은 특징 담음:
 - 처음: 채널 4 (기본 정보만)
 - 첫 번째 Conv 후: 채널 16
 - 두 번째 Conv 후: 채널 256
3. **이미지 임베딩에 더하기**

이 마스크 특징(256채널)을 사진 특징(이미지 임베딩, 256채널)과 같은 크기로 맞춰서 픽셀별로 더함

→ 사진+마스크 힌트가 합쳐진 새 지도 생성

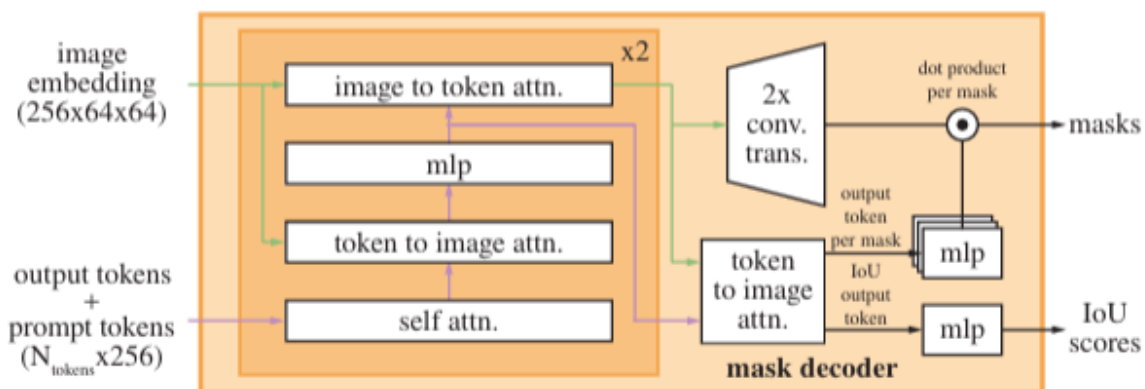
4. 만약 마스크 없으면? → no-mask라는 고정된 숫자(256채널짜리)를 대신 더함

= "마스크 힌트 없음"이라는 신호를 주는 것

3) Mask Decoder

지금까지 이미지 임베딩(사진 분석 지도)과 프롬프트 토큰(클릭, 박스 등 힌트)을 준비했고 둘을 섞어서 마스크(색칠 영역)를 만드는 단계 = Mask Decoder

디코더 구성



SAM의 디코더는 같은 구조의 층을 2번 반복

1. 토큰끼리 정보 교환 (*self-attention*)

- 프롬프트 토큰과 출력 토큰들이 서로 attention하며 서로 좌표, 범위 정보 교환

2. 토큰이 이미지를 질문 (*token → image cross-attention*)

- 토큰들이 이미지 임베딩 지도 속에서 자기한테 필요한 정보만 서치
- ex. 이 클릭 좌표 근처의 모양이 어떤지 알려줘

3. 토큰 내부 계산 (*MLP*)

- 토큰 속 숫자(정보)를 조합·변형해 더 유용하게 변경

4. 이미지가 토큰을 질문 (*image → token cross-attention*)

- 이미지 임베딩이 토큰들 참고

- ex. 이 픽셀 위치가 마스크 안인지 바깥인지, 클릭/박스 정보 좀 알려줘
- ⇒ 이렇게 하면 프롬프트 정보 ↔ 이미지 정보가 두 방향으로 충분히 섞임

마스크 생성

두 층을 거친 뒤 마스크를 다음 4단계로 생성:

1. 이미지 해상도 복원

- 현재 이미지 임베딩은 원본보다 1/4 크기(64×64)
 - 전치합성곱(Transpose Conv) 두 번으로 4배 키워서 원래 크기에 가깝게 복원

2. 출력 토큰으로 분류기 만들기

- 출력 토큰을 작은 MLP(3층짜리)에 넣어서 동적 분류기 벡터 생성 (256차원짜리)

3. 픽셀별 비교

- 복원된 이미지 임베딩의 각 픽셀(256차원)과 출력 토큰 분류기 벡터를 내적(dot product)해서
해당 픽셀이 전경(물체 안)일 확률 점수를 계산

4. 시그모이드

- 점수를 0~1 사이 확률로 변환 (픽셀마다 안 / 밖 판정) → 픽셀별 확률맵(마스크) 완성

4) 학습 세부

- 손실
 - Focal + Dice (20:1 비율)
 - IoU 예측 헤드: MSE → 마스크 손실에 가중치 1.0으로 합산
- 프롬프트 시뮬레이션
 - 한 마스크당 11라운드
 - 1라운드: 점/박스 중 랜덤
 - 이후: 이전 예측과 GT의 차이 영역에서 점 추가
 - 로짓을 넘겨 정보 최대 보존

- **효율성**

- 이미지 임베딩은 한 번만 계산
- 디코더 FLOPs는 인코더의 1% 미만 → CPU 브라우저에서도 ~50ms 반응

4 Segment Anything Data Engine

SAM을 학습하려면 매우 다양한 물체와 장면에 대한 마스크 데이터 필요

but 기존 공개 데이터셋들은 범위가 한정적이고 수가 적음

⇒ 그래서 **자동+반자동 방식으로 11억 개 이상의 마스크**를 생성하는 데이터 엔진을 만듦

4.1 전체 파이프라인 흐름

크게 3단계로 진행

각 단계마다 사람의 개입 정도가 다르고, 자동화 비율이 점점 높아짐

Step 1. Assisted Manual Annotation (반자동 주석)

사람이 사진에서 물체 클릭하거나 박스 지정 → SAM이 초기 마스크 예측

- 사람이 틀린 부분만 수정
- 순수 수작업보다 훨씬 빠름
- 초기에 모델이 약하니까 사람 개입이 많음

Step 2. Semi-Automatic Annotation (반자동 확장)

1단계에서 만든 마스크로 SAM을 조금 학습 → 성능이 올라감

- 한 이미지에서 일부 객체만 수동 클릭 → 나머지 비슷한 객체는 SAM이 자동 분할
- 사람이 자동 생성 마스크를 훑어보고 필요하면 수정
- 사람 개입이 크게 줄어듦

Step 3. Fully Automatic Annotation (완전 자동)

- 모델이 충분히 학습되면 사람 없이 이미지 전체 자동 분할
- 이미지에 나올 수 있는 모든 객체 후보 생성하고 신뢰도 낮은 건 필터링
- 대규모 데이터가 폭발적으로 늘어남

4.2 프롬프트 시뮬레이션

SAM은 클릭·박스 같은 프롬프트를 받아야 마스크 생성 가능

but 모든 이미지에 사람이 직접 프롬프트를 입력하는 건 불가능 → 가짜 프롬프트 만들어서 학습

- **점 프롬프트:** GT 마스크 내부에서 랜덤 점 추출 (전경 점, 배경 점 모두 생성)
- **박스 프롬프트:** GT 마스크를 감싸는 최소 직사각형 생성
- **마스크 프롬프트:** 이전 단계에서 예측한 마스크를 그대로 입력으로 사용

4.3 품질 관리

자동 생성 마스크 중 너무 작거나 이상한 모양은 필터링

IoU, 마스크 면적, 픽셀 분포 등 통계로 품질 점검, 일부 샘플은 사람이 다시 검토

4.4 규모와 성과

최종적으로 11억 개 이상의 마스크, 1,100만 장 이상의 이미지 확보

이 방대한 데이터로 학습한 SAM은 **범용 제로샷 성능** 갖추게 됨

5 Segment Anything Dataset

5.1 SA-1B 데이터셋

- **규모:**
 - 11억 개 마스크
 - 1,100만 장 이미지
- **형식:**
 - 이미지 파일(.jpg/.png) + 각 객체의 마스크 데이터(.json 등)
- **특징:**
 - 범용성을 위해 전 세계 다양한 장면 포함
 - 사람, 동물, 사물, 배경 등 범주 제한 없음

- 모든 마스크는 픽셀 단위로 정확히 구분됨

5.2 데이터 수집 과정

1. 초기 수집

- 다양한 도메인의 이미지 크롤링 (웹, 오픈데이터, 라이선스 이미지 등)
- 중복/저해상도/저품질 이미지 제거

2. 어노테이션(마스크 생성)

- Step 1~3 (사람+SAM 협력 → 완전 자동) 방식으로 마스크 생성
- 품질 필터링 적용:
 - 마스크 크기 너무 작거나 크면 제외
 - IoU, 픽셀 분포 등 품질 지표 검사

3. 프롬프트 시뮬레이션

- 학습 시 다양한 입력(점, 박스, 마스크)을 쓸 수 있도록 각 마스크마다 가짜 프롬프트 생성

5.3 데이터 다양성 확보 방법

- 도메인 다양화: 풍경, 실내, 스포츠, 제품, 위성사진, 의료 이미지 등
- 객체 크기 다양화: 아주 작은 물체부터 큰 배경까지 포함
- 복잡도 다양화: 단순 배경 vs 복잡한 배경, 단일 객체 vs 다중 객체
- 지역 편향 최소화: 다양한 국가·문화권 이미지 사용

6 Segment Anything RAI Analysis

SAM은 범용 분할 모델이라서 다양한 분야에 사용될 수 있는데, **프라이버시 침해, 편향 강화, 악의적 사용** 가능성 있음

특히, 학습 데이터가 방대하고 인터넷 이미지가 포함되기 때문에 **데이터 편향**이나 **저작권 문제**도 발생

1. 프라이버시

- 이미지 안에 사람 얼굴, 차량 번호판 등 개인 식별 가능 정보(PII)
- 이런 정보가 포함된 채로 학습되면 모델이 해당 세부 정보를 복원하거나 악용할 가능성 존재

2. 데이터 편향

- 이미지 소스가 특정 국가·문화·환경에 편중될 수 있음
- 객체 종류, 배경, 색상, 촬영 환경 등에서도 편향 발생 가능

3. 악의적 사용

- 무기·불법 활동·감시 등 유해 목적으로 악용될 수 있음.

4. 법적·저작권 문제

- 데이터셋 이미지 출처가 웹인 경우 저작권이 있는 이미지가 포함될 수 있음
- 상업적 사용 시 법적 문제

완화 전략

- 데이터 필터링: 얼굴·번호판 자동 블러 처리, 민감 장면 제거
- 사용 가이드라인: 모델·데이터 사용 허가 범위와 제한 명시
- 편향 모니터링: 다양한 도메인에서 성능 평가 후 편향 수정
- 윤리 검토 프로세스: 학습·배포 전 RAI 점검 절차 도입

7 Zero-Shot Transfer Experiments

SAM이 다른 데이터로 재학습(fine tuning) 없이 학습된 상태 그대로 새로운 작업에 적용했을 때 얼마나 잘 작동하는지 실험

= 제로샷(Zero-Shot) 성능 평가

7.1 실험 목적

학습에 없던 데이터셋과 과제에서 얼마나 성능이 나오는지 검증

특히 도메인 전이 능력을 확인:

- 일반 사진 → 의료 영상

- 고화질 카메라 → 저화질 CCTV
- 일상 사물 → 특수 분야(위성, 현미경 등)

7.2 평가 방법

학습 시 사용하지 않은 다양한 공개 데이터셋에서 테스트

- **프롬프트 방식:** 점, 박스, 마스크 등 여러 형태 입력 사용
- **비교 대상:** 다른 SOTA 분할 모델
- **지표:** mIoU (Mean Intersection over Union), Dice score 등

7.3 주요 결과

1. 일반 객체 분할 (Generic Object Segmentation)

- COCO, ADE20K 같은 범용 데이터셋에서 평가
- 별도 학습 없이도 **기존 SOTA와 비슷하거나 더 나은 성능**
- 특히 박스나 클릭 프롬프트 기반 분할에서 강점

2. 특수 도메인 전이

- **의료 영상:** CT, MRI 같은 데이터에 점 프롬프트 몇 개로도 좋은 성능
- **위성 사진:** 건물·도로 분할에 적용 가능
- **수중 영상:** 어류, 해양 생물 분할 가능

3. 고정밀 필요 작업

- 복잡한 장면에서도 클릭 몇 번으로 다양한 마스크 제안
- 다중 마스크 출력 기능 덕분에 모호한 경우도 선택 폭 넓음

7.4 분석

SAM이 범용적으로 잘 동작하는 이유:

1. 초거대 데이터셋 (SA-1B) 덕분에 다양한 시각 패턴 학습
2. 프롬프트 기반 구조, 작업 규칙을 새로 학습할 필요 없음
3. 이미지 인코딩이 고품질, 도메인이 달라도 시각적 특징 잘 추출

한계:

- 세밀한 경계(특히 투명 객체, 머리카락 등)에서 정밀도 떨어짐
- 훈련 데이터에 전혀 없는 시각 패턴은 여전히 어려움